

Two-Factor RFID + Face Recognition Door

Low-Level Design, Interfaces & Implementation Contract

Document ID	DOC-03	Status	Implementation Baseline
-------------	--------	--------	-------------------------

Implementation rule: This document is the direct handoff to the implementer or AI coding agent. Requirements are normative; design choices are the default implementation; deviations must preserve requirement IDs and acceptance behavior.

1. Repository / Module Layout

```

two-factor-door/
├── app/
│   ├── domain/
│   │   ├── models.py
│   │   ├── states.py
│   │   ├── decisions.py
│   │   └── reason_codes.py
│   ├── application/
│   │   ├── access_controller.py
│   │   ├── enrollment_service.py
│   │   ├── health_service.py
│   │   └── event_logger.py
│   ├── ports/
│   │   ├── rfid.py
│   │   ├── camera.py
│   │   ├── recognizer.py
│   │   ├── device_transport.py
│   │   ├── repositories.py
│   │   ├── model_store.py
│   │   └── clock.py
│   ├── adapters/
│   │   ├── esp32_serial.py
│   │   ├── opencv_camera.py
│   │   ├── opencv_lbph.py
│   │   ├── sqlite_repositories.py
│   │   ├── file_model_store.py
│   │   └── monotonic_clock.py
│   ├── config.py
│   └── main.py
├── firmware/
│   ├── src/main.cpp
│   └── protocol.h
├── data/
│   ├── app.db
│   ├── faces/<person_id>/sample_*.png
│   └── models/lbph.yml

```

```

|   └─ logs/access.jsonl
└─ protocol/
|   └─ schema.json
└─ tests/
|   └─ unit/
|   └─ integration/
|   └─ hardware/
└─ docs/

```

2. Domain Model

Object	Responsibilities	Key fields
Person	Represents an enrolled identity	person_id, display_name, face_label, status
RFIDCard	Binds a physical card UID to a person	uid, person_id, status
RecognitionResult	Carries model output without granting access	label, distance, captured_at
AccessSession	Tracks one RFID-driven verification attempt	session_id, card_uid, expected_person_id, started_at, deadline, state, match_count
AccessDecision	Final authorization result	decision, reason, person_id, session_id
SystemConfig	Validates required runtime settings	window, hold, threshold, match count, ports

3. Interfaces / Ports

```

class CameraPort(Protocol):
    def open(self) -> None: ...
    def read(self) -> Frame | None: ...
    def close(self) -> None: ...

class FaceRecognizerPort(Protocol):
    def predict(self, face_gray: ArrayLike) -> RecognitionResult: ...
    def train(self, labeled_faces: Sequence[tuple[int, ArrayLike]]) -> None: ...
    def save(self) -> None: ...
    def load(self) -> None: ...

class DeviceTransportPort(Protocol):
    def send_access_command(self, command: DeviceCommand) -> None: ...
    def poll(self) -> list[DeviceEvent]: ...
    def health(self) -> TransportHealth: ...

```

```

class IdentityRepository(Protocol):
    def get_card_owner(self, uid: str) -> Person | None: ...
    def get_person(self, person_id: str) -> Person | None: ...
    def create_person(self, person: Person) -> None: ...
    def bind_card(self, uid: str, person_id: str) -> None: ...

class EventRepository(Protocol):
    def append(self, event: AccessEvent) -> None: ...

class ClockPort(Protocol):
    def now_monotonic(self) -> float: ...

```

4. Core Authorization Policy

```

def authorize(session, recognition, now, config):
    if now >= session.deadline:
        return DENY(TIMEOUT)

    if recognition is MULTIPLE_FACES:
        return DENY(MULTIPLE_FACES)

    if recognition is NONE:
        return CONTINUE

    if recognition.distance > config.lbph_threshold:
        return CONTINUE

    if recognition.person_id != session.expected_person_id:
        return DENY(FACE_MISMATCH)

    session.match_count += 1
    if session.match_count >= config.required_consistent_matches:
        return GRANT(MATCHED_OWNER)

    return CONTINUE

```

Important: RecognitionResult must be data. It must not call the servo, LEDs, buzzer, serial adapter, or database.

5. AccessController Pseudocode

```

loop:
    events = device_transport.poll()

    for event in events:
        if event.type == RFID_DETECTED:
            begin_or_reject_session(event)

```

```

if session.state == VERIFYING:
    if clock.now_monotonic() >= session.deadline:
        deny(TIMEOUT)
    else:
        frame = camera.read()
        faces = detector.detect(frame)

        if len(faces) == 0:
            continue
        if len(faces) > 1:
            deny(MULTIPLE_FACES)
        else:
            result = recognizer.predict(normalize(faces[0]))
            decision = policy.evaluate(session, result)
            execute_decision(decision)

```

6. Decision Execution

GRANT

- > persist AccessEvent(GRANTED)
- > device.send_access_command(decision=grant, hold_ms=config.door_hold_s)
- > wait for command_result
- > if execution fails: log ACTUATOR_FAILURE; keep safe state

DENY / TIMEOUT / FAULT

- > persist AccessEvent(result)
- > device.send_access_command(decision=deny, indication=reason)
- > remain closed

7. NDJSON Protocol Contract

Envelope

```

{
  "v": 1,
  "type": "...",
  "event_id": "...", // event messages
  "session_id": "...", // access workflow correlation
  "command_id": "...", // device command correlation
  "device_id": "door-01",
  "payload": {}
}

```

RFID_DETECTED payload:

```

{"uid":"A1B2C3D4"}

```

ACCESS_COMMAND payload:

```

{"decision":"grant|deny","reason":"MATCHED_OWNER|FACE_MISMATCH|TIMEOUT|...", "door_hold_ms":3000}

```

COMMAND_RESULT payload:

```
{"result": "executed | rejected | error", "error_code": null}
```

HEARTBEAT payload:

```
{"state": "CLOSED | OPEN | FAULT"}
```

8. Device Firmware State Machine

DEVICE_STARTUP

- > CLOSED_SAFE

- > IDLE

IDLE + RFID card -> emit RFID_DETECTED -> remain IDLE

IDLE + ACCESS_COMMAND(grant)

- > validate command/session freshness

- > OPENING

- > OPEN_HOLD

- > CLOSING

- > IDLE

IDLE + ACCESS_COMMAND(deny)

- > DENY_INDICATION

- > IDLE

Any invalid/stale/duplicate command

- > ignore or reject

- > CLOSED_SAFE

9. Hardware Pin Baseline

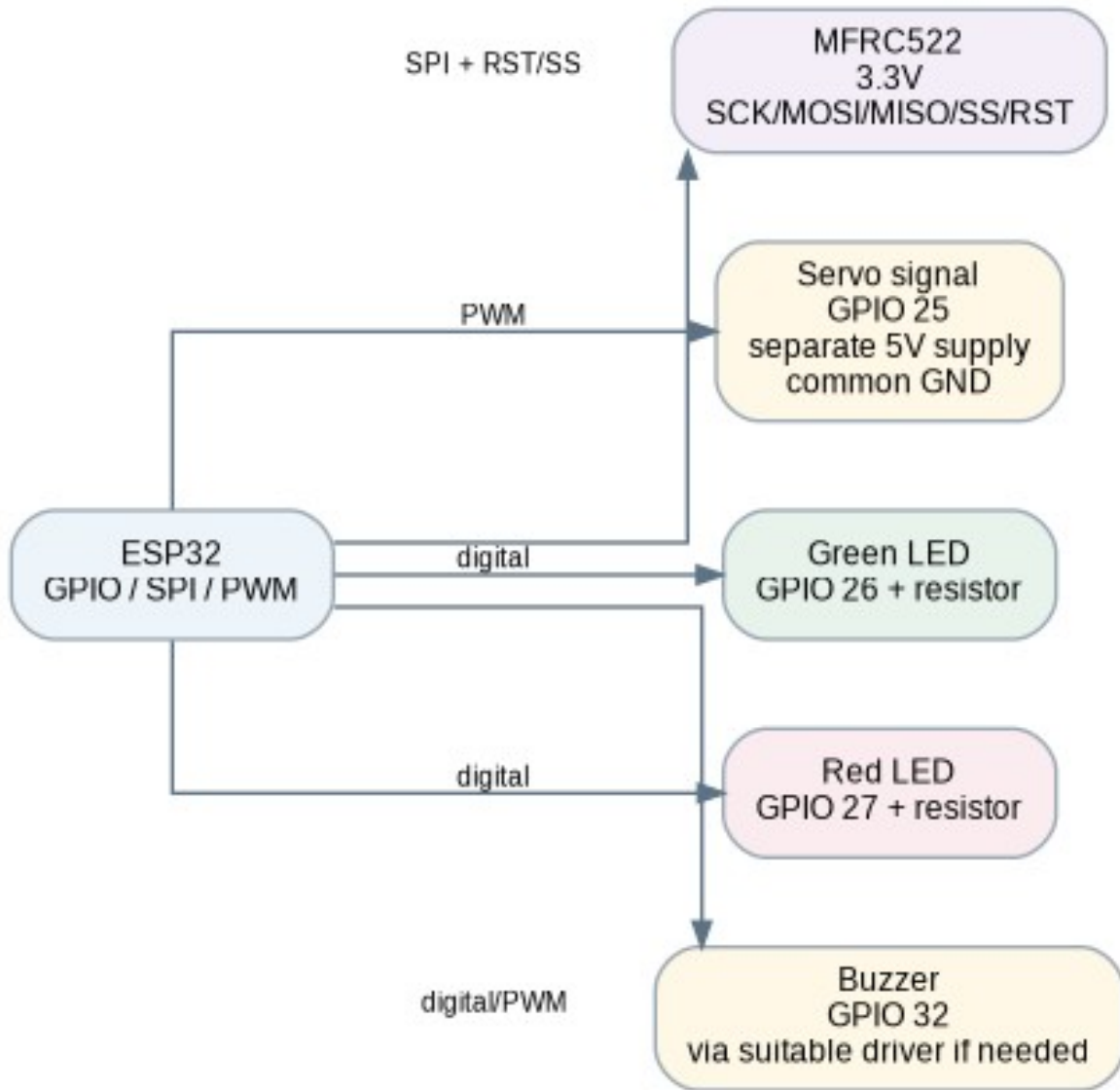


Figure 3. Proposed example pin map; verify against the exact ESP32 board before wiring

Component	Signal	ESP32 pin	Notes
MFRC522	SCK	18	SPI clock
MFRC522	MISO	19	SPI data to ESP32
MFRC522	MOSI	23	SPI data from ESP32
MFRC522	SS/SDA	5	SPI chip select
MFRC522	RST	22	Reader reset
MFRC522	3.3V/GND	3V3/GND	Keep logic at supported voltage
Servo	Signal	25	PWM signal; servo should

Component	Signal	ESP32 pin	Notes
			use suitable external power
Green LED	Anode via resistor	26	Digital output
Red LED	Anode via resistor	27	Digital output
Buzzer	Control	32	Digital or PWM depending on buzzer type; use driver if required

Board note: GPIO choices are a proposed baseline for a common ESP32 development board, not a universal wiring law. Verify pin availability/strapping conflicts on the exact board revision before assembly.

10. Face Processing Pipeline

```

capture frame
-> grayscale (for LBPH)
-> face detection
-> exactly-one-face gate
-> crop face region
-> normalize dimensions / optional histogram equalization
-> LBPH.predict()
-> threshold check
-> identity binding check
-> stable-match gate

```

OpenCV documentation states that LBPH expects grayscale image input and exposes a threshold-based rejection mechanism. The concrete detection method is an implementation choice; a lightweight local detector is sufficient for this MVP.

11. Enrollment Pipeline

```

enroll_person(person_id):
  capture until sample_count >= configured minimum
  for each frame:
    faces = detector.detect(frame)
    accept only exactly one usable face
    save normalized grayscale sample
  train LBPH on labeled samples
  persist model + label map
  bind RFID UID -> person_id
  run live verification test

```

12. Configuration

```
verification_window_s: 10

door_hold_s: 3
required_consistent_matches: 3
lbph_threshold: CALIBRATE_FROM_ACTUAL_DATA
camera_index: 0
serial_baud: 115200
heartbeat_interval_s: 1
heartbeat_timeout_s: 3
allow_multiple_faces: false
```

The LBPH threshold is not hard-coded by this document; it must be calibrated from the actual dataset and webcam, then recorded as a versioned configuration value.

13. Error / Reason Codes

Code	Meaning	Action
UNKNOWN_RFID	UID not enrolled/active	Deny
FACE_NOT_FOUND	No face in frame	Continue until timeout
MULTIPLE_FACES	Ambiguous frame	Deny
FACE_MISMATCH	Known face is not card owner	Deny
LOW_CONFIDENCE	LBPH distance above threshold	Continue until timeout
TIMEOUT	10-second window expired	Deny
CAMERA_ERROR	Webcam unavailable	Fault/deny
DEVICE_LINK_ERROR	ESP32 communication unavailable	Fault/deny
COMMAND_REJECTED	ESP32 rejected command	Fault/deny; log
ACTUATOR_FAILURE	Expected device action failed	Log; maintain safe state

14. Test Pyramid

Level	Examples	Gate
Unit	Policy decisions, state transitions, timer expiration, protocol parsing, repository rules	Every requirement in policy covered.
Integration	Camera + detector + LBPH, SQLite +	Deterministic flows work end-to-end

Level	Examples	Gate
	enrollment, serial adapter + firmware simulator	on laptop.
Hardware-in-loop	Real MFRC522, real cards, real servo/LED/buzzer, webcam	ACC-001..ACC-010 pass under controlled conditions.
Failure injection	Unplug camera, unplug serial, send malformed command, reset ESP32	Door remains closed and fault is observable.

15. Implementation Constraints for an AI Coding Agent

- Implement requirement IDs exactly as written; do not invent product features to fill perceived gaps.
- Do not introduce network services, brokers, cloud APIs, or microservices for the MVP.
- Do not let recognition code access GPIO or serial directly; use ports/adapters.
- Do not let firmware make biometric or identity decisions.
- Do not grant access on one weak signal; the canonical authorization rule must remain unchanged.
- Do not silently change the 10-second window, fail-closed rule, or wrong-face behavior.
- Every change that affects behavior must update the relevant requirement/test mapping.
- Prefer small deterministic modules with tests over framework-heavy abstractions.

16. Definition of Done

Area	Done when
Requirements	All FR/NFR/SEC requirements traced to implementation and at least one test.
Software	Host starts, initializes dependencies, runs state machine, persists identities/events, and exposes useful logs.
Firmware	ESP32 reads RFID, accepts versioned commands, drives outputs, and fails safe.
Face recognition	Enrollment and live recognition work with actual webcam and calibrated threshold.
Integration	Correct path and all negative paths in ACC-001..ACC-010 work.
Safety	Power cycle, serial loss, camera loss, malformed command, and restart leave the door closed.
Documentation	Known photo-spoof and RFID-cloning limitations are visible in project docs.

17. Reference Notes

OpenCV LBPHFaceRecognizer supports configurable radius, neighbors, grid dimensions, and threshold; its documented API also states that the LBPH features expect grayscale images. NXP currently lists the MFRC522 as End of Life / not recommended for new designs, which is acceptable here only because the part is a fixed project constraint for a prototype.